SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Molabanti Varun Sai	**Reg. No.:**	192425247
Date:	26/10/2026	**Duration:**	60 minutes

Code of Conduct

I M.Varun Sai (192425247) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Varun Sai

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

1. Analyze System Requirements

1.1 System Objectives

The main objective is to develop a centralized software platform that connects consumers, collection centres, logistics providers, recyclers, and administrators to manage the complete e-waste lifecycle.

The system aims to:

- Manage e-waste collection requests and pickup scheduling.
- Track e-waste during collection, transportation, and recycling.
- Maintain classification, weight, and recycling records.
- Generate sustainability dashboards and reports.
- Provide secure communication through APIs.
- Support scalable deployment, monitoring, logging, and automated testing.

1.2 Functional Requirements

	quirement	**chitectural Impact**
1	er registration and login	thentication component
2	omit collection requests	llection management service
3	nedule and assign pickups	neduling and logistics service
4	ack e-waste movement	acking service
5	assify and weigh materials	assification component
6	cord recycling and recovery	cycling management service
7	nerate sustainability reports	alytics and reporting component

8	ınage users and permissions	er management and access control
9	ıd notifications	tification service
10	nitor errors and service health	nitoring and logging infrastructure

1.3 Non-Functional Requirements

- Scalability: Support growth in users, collection requests, and recycling records.
- Maintainability: Allow individual modules to be modified independently.
- Reliability: Maintain accurate records and prevent data loss.
- Availability: Keep core services operational with minimal downtime.
- Performance: Process requests and display tracking information efficiently.
- Security: Protect user information and restrict unauthorized access.
- Usability: Provide simple interfaces for different stakeholder roles.
- Interoperability: Integrate with notification, mapping, and external reporting systems.

1.4 Key Quality Attributes

The architecture must balance security, performance, scalability, and reliability while keeping development manageable for an academic project.

2. Select a Suitable Software Architecture

Selected Architecture: Hybrid Layered Architecture with Modular Services

The proposed system uses a Hybrid Architecture combining:

1. Layered Architecture for separating presentation, business logic, and data access.
2. Client–Server Architecture for communication between users and the backend.
3. Modular Services for collection, scheduling, tracking, recycling, and analytics.
4. Event-Driven Communication for notifications and asynchronous background tasks.

Why is this architecture suitable?

- It separates user interfaces from business logic and database operations.
- Each business module has clearly defined responsibilities.
- Secure APIs allow different stakeholders and external systems to communicate.
- Modules can be tested and maintained independently.
- Background processing helps notifications and report generation avoid slowing down normal requests.
- The architecture can start as a modular monolith and evolve into independently deployed microservices if system demand increases.

For the initial academic implementation, a modular backend with clear service boundaries is recommended instead of deploying many separate microservices immediately. This reduces infrastructure complexity while preserving future scalability.

3. Software Architecture Diagram

The following diagram illustrates the major components, databases, external integrations, and communication paths.

Diagram note: Solid arrows represent normal application interactions. Dashed arrows indicate optional integrations or infrastructure operations. In an initial implementation, the business modules may run within one backend application and share a relational database.

4. Explain Components and Their Interactions

4.1 Major Architectural Components

mponent	rpose and Responsibilities
sentation Layer	ovides web pages and dashboards for consumers, collection centres, logistics providers, recyclers, and administrators.
I Layer	ceives HTTP requests, routes them to appropriate backend modules, and returns responses in JSON format.
thentication and RBAC	rifies user identity and checks permissions based on stakeholder roles.
er Management	res user profiles, manages accounts, and supports account administration.
llection Management	eates, updates, and maintains e-waste collection requests.
kup Scheduling	nages available slots, pickup assignments, and scheduling conflicts.
cking and Logistics	cords collection, transportation, delivery, and location updates.
ssification Module	cords e-waste categories, quantities, weights, and conditions.
cycling Management	intains processing records and details of recovered materials.
alytics and Reporting	lculates collection totals, recycling quantities, recovery rates, and sustainability indicators.
tification Module	nds confirmations and status updates through email or SMS integrations.
tabase Layer	res users, requests, schedules, material batches, transport records, and recycling data.
nitoring and Logging	cks application errors, performance, system health, and security-relevant activities.
CD and Containers	tomates builds, testing, packaging, and deployment.

4.2 Component Communication

The components communicate using the following mechanisms:

- HTTPS and REST APIs: Transfer requests between the frontend and backend.
- JSON: Exchange structured request and response data.
- Database queries: Read and update persistent records.
- Events or background jobs: Process notifications and resource-intensive reports asynchronously.
- External APIs: Connect to email, SMS, mapping, or GPS services when configured.
- Monitoring endpoints and logs: Report service health, errors, and performance metrics.

For example, when a consumer submits a collection request, the API validates the request, checks authentication and permissions, and passes it to the collection management module. The module

stores the request in the database and returns a unique request ID. A notification can then be sent through a background job.

4.3 Overall System Workflow

Workflow explanation:

1. The consumer logs in and submits an e-waste collection request.
2. The system validates the details and creates a unique request record.
3. The consumer selects a pickup slot.
4. The administrator or scheduling module assigns a logistics provider.
5. The logistics provider collects and transports the e-waste.
6. The collection centre records the material category, weight, and condition.
7. The recycler records processing activities and recovered materials.
8. The database maintains the lifecycle records.
9. The analytics module generates sustainability reports.
10. Notifications inform relevant stakeholders about important status changes.

5. Discuss Quality Attributes

5.1 Scalability

Design approach:

- Keep business modules independent.
- Use stateless API handlers where practical.
- Add backend instances behind a load balancer when required.
- Optimize database queries and indexes.
- Use caching for frequently accessed, non-sensitive data where appropriate.
- Move heavy reporting and notifications to background processing.

Benefit: The platform can handle increasing collection requests and stakeholder activity without redesigning the entire system.

5.2 Security

Design approach:

- Use HTTPS for network communication.
- Apply secure password hashing and authentication.
- Implement role-based access control (RBAC).
- Validate and sanitize user input.
- Use parameterized database queries.
- Protect APIs using authorization checks and rate limits.
- Store secrets outside application source code.
- Maintain audit records for sensitive operations.

Benefit: User information, collection records, and recycling data are protected against unauthorized access and common application vulnerabilities.

5.3 Performance

Design approach:

- Use database indexes for request IDs, statuses, and frequently searched fields.
- Retrieve only the data required by each page.

- Paginate large tracking histories and reports.
- Cache suitable read-heavy information.
- Process report generation and notifications asynchronously when necessary.

Benefit: The system can provide responsive interfaces and process larger datasets efficiently.

5.4 Reliability

Design approach:

- Use database transactions for related record updates.
- Validate state transitions throughout the e-waste lifecycle.
- Back up the database and test restoration procedures.
- Handle API failures with appropriate error messages and retry policies.
- Prevent duplicate requests or repeated background jobs where possible.
- Maintain audit history for important changes.

Benefit: The system reduces inconsistent records, duplicate processing, and data loss.

5.5 Maintainability

Design approach:

- Separate business logic into well-defined modules.
- Use consistent naming, coding standards, and API contracts.
- Maintain unit, integration, and API tests.
- Use version control and code reviews.
- Document database schemas and interfaces.
- Automate builds and deployment.

Benefit: Developers can update collection, tracking, or reporting functionality with less risk of affecting unrelated modules.

5.6 Availability

Design approach:

- Use health checks and application monitoring.
- Deploy multiple backend instances if required by the service target.
- Configure automated recovery and database backups.
- Use rolling deployments where supported.
- Monitor external service failures and provide fallback behavior where practical.

Benefit: Failures can be detected and recovered from more quickly, reducing service interruptions.

Availability is not guaranteed by the architecture alone; it also depends on infrastructure, deployment configuration, backup practices, and operational support.

6. Justify Architectural Decisions

6.1 Rationale for the Selected Architecture

The hybrid layered architecture is selected because the platform contains multiple business functions that must share information while maintaining clear separation of responsibilities.

chitectural Decision	**stification**
yered design	parates the user interface, business logic, and data access.
ent–server	ows different stakeholder groups to access the platform through a

communication	browser.
odular business services	eps collection, scheduling, tracking, and recycling logic organized.
ST APIs	pport secure communication and future integration with external systems.
lational database	intains structured records and relationships between requests, batches, and recycling activities.
ent-driven background jobs	ow notifications and report generation to run without blocking user requests.
ntainerization	kes deployment more consistent across development and production environments.
CD pipeline	tomates testing and deployment to reduce manual errors.
ntralized monitoring	lps detect performance issues, errors, and service failures.

6.2 Architectural Trade-Offs

vantage	ade-Off
odular design improves maintainability.	quires clear module boundaries and interface definitions.
shared relational database simplifies transactions.	avy workloads can create database bottlenecks.
ckground jobs improve responsiveness.	quire job status tracking, retries, and failure handling.
ntainers simplify deployment.	roduce configuration and orchestration overhead.
nodular monolith simplifies initial development.	lependent module scaling is more limited than with separately deployed services.
ST APIs enable integration.	quire API versioning, access control, and error handling.
nitoring improves fault detection.	ds storage, configuration, and operational costs.

6.3 Future Enhancements

The selected architecture supports future development in the following areas:

1. AI-based classification: Add image recognition to identify electronic devices and suggest material categories.
2. GPS integration: Track collection vehicles and optimize pickup routes.
3. IoT integration: Monitor storage capacity and selected environmental conditions at collection centres.
4. Blockchain-based traceability: Explore tamper-evident records for material movement and recycling certificates.
5. Mobile application: Provide dedicated interfaces for consumers and collection agents.
6. Microservices migration: Separate high-demand modules, such as tracking or analytics, into independent services when justified by operational needs.
7. Advanced analytics: Predict e-waste generation and estimate recycling and material

recovery trends.

8. External integrations: Connect with authorized recycling partners and environmental reporting systems.

6.4 Long-Term Maintainability and Integration

The architecture supports long-term maintenance by keeping business logic separate from presentation and persistence concerns. Documented API contracts allow external applications to integrate without directly accessing the internal database.

A modular implementation can initially run as one deployable backend. As traffic, development-team size, or operational requirements grow, selected modules can be extracted into independent services. This incremental approach avoids the cost and complexity of introducing a full microservices infrastructure before it is needed.

Conclusion

The Intelligent E-Waste Collection and Sustainable Recycling Platform is best supported by a hybrid layered architecture with modular business services, secure REST APIs, a relational database, and optional event-driven background processing.

This design provides a practical balance between scalability, security, performance, reliability, availability, and maintainability. It supports the complete e-waste lifecycle, from collection requests to recycling records and sustainability reporting, while allowing future integration with AI, GPS, IoT, and external environmental systems.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient	Diagram is incomplete, unclear, or technically incorrect.
	interfaces, and interactions accurately represented.		detail.	
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25